

Practical : 6	Implementation of Chain Matrix Multiplication using Dynamic Programming.
Date : 12/08/2026	

Objective :- To implement the Matrix Chain Multiplication problem in C++ using Dynamic Programming to find the parenthesization that minimises the number of scalar multiplications, and to analyse its time and space complexity.

Program :-

```
#include <iostream>
#include <vector>
#include <climits>
using namespace std;

// ----- Matrix Chain Multiplication (DP,  $O(n^3)$ ) -----
int matrixChainOrder(vector<int>& p, int n, vector<vector<int>>& split) {
    vector<vector<int>> dp(n, vector<int>(n, 0));

    for (int len = 2; len < n; len++) {
        for (int i = 1; i < n - len + 1; i++) {
            int j = i + len - 1;
            dp[i][j] = INT_MAX;
            for (int k = i; k < j; k++) {
                int cost = dp[i][k] + dp[k + 1][j] + p[i - 1] * p[k] * p[j];
                if (cost < dp[i][j]) {
                    dp[i][j] = cost;
                    split[i][j] = k;
                }
            }
        }
    }
    return dp[1][n - 1];
}

void printParens(vector<vector<int>>& split, int i, int j) {
    if (i == j) {
        cout << "A" << i;
        return;
    }
    cout << "(";
    printParens(split, i, split[i][j]);
    printParens(split, split[i][j] + 1, j);
    cout << ")";
}

int main() {
    // dimensions: matrices A1..A5 with p = {40, 20, 30, 10, 30, 5}
    vector<int> p = {40, 20, 30, 10, 30, 5};
    int n = p.size();
}
```

```
vector<vector<int>> split(n, vector<int>(n, 0));

int minOps = matrixChainOrder(p, n, split);

cout << "Minimum number of multiplications: " << minOps << endl;
cout << "Optimal Parenthesization: ";
printParens(split, 1, n - 1);
cout << endl;
return 0;
}
```

Output :-

Minimum number of multiplications: 10000
Optimal Parenthesization: (A1(A2(A3(A4A5))))

Analysis:-

Time & Space Complexity (n = number of matrices in the chain):

Method	Time Complexity	Space Complexity
Matrix Chain - Dynamic Programming	$O(n^3)$	$O(n^2)$
Brute Force (all parenthesizations)	$\Omega(4^n / n^{1.5})$	$O(n)$

Matrix Chain Multiplication finds the parenthesization of a chain of matrices that minimises the total number of scalar multiplications needed to compute their product; the result of the multiplication is fixed, but the number of operations depends heavily on grouping. The DP table $dp[i][j]$ holds the minimum cost of multiplying matrices $A_i..A_j$, computed by trying every possible split point k and combining the cost of the two sub-chains with the cost $p[i-1]*p[k]*p[j]$ of the final multiplication; there are $O(n^2)$ such (i, j) pairs and each takes $O(n)$ time to minimise over k , giving $O(n^3)$ time overall, with $O(n^2)$ space for the DP and split tables. A brute-force search over every possible parenthesization instead grows following the Catalan numbers, which is exponential in n , making the DP approach dramatically faster for longer chains. For the sample dimensions $p = \{40, 20, 30, 10, 30, 5\}$ (five matrices $A_1..A_5$), the DP finds that grouping as $(A_1(A_2(A_3(A_4A_5))))$ minimises the total multiplications at 10000, far fewer than a naive left-to-right evaluation would require.

Conclusion :- Matrix Chain Multiplication was successfully implemented in C++ using bottom-up Dynamic Programming over increasing chain lengths, with a split table used to reconstruct the optimal parenthesization. The program correctly found the minimum multiplication cost of 10000 and the optimal grouping $(A_1(A_2(A_3(A_4A_5))))$ for the sample dimensions, confirming the expected $O(n^3)$ time and $O(n^2)$ space behaviour. This is a substantial improvement over the exponential cost of evaluating every possible parenthesization by brute force, and illustrates the classic DP technique of solving optimally over increasing sub-problem sizes.